Andres Garcia Barros ID: 921107769
Github: AmgBar CSC615 Embedded Linux

Assignment 4 – Follow the line but stop

Description:

Using the IR Obstacle Sensor and the Line sensor, their input will be read in order tell tell the user whether the sensors are over Black or white and the other sensor will be used to tell the user if there is an obstruction in front of the sensor or not. This will be done using threads, one thread for each sensor and a main loop that checks values.

Approach / What I Did:

To help me complete this assignment, I checked an old assignment from CSC 415, where I used threads, specifically in the police records csv assignment, where I used threads to tally up the number of crimes of certain types into a table.

From that assignmen,t I got a refresher on how to use threads and the way I figured how to use threads was by making sure to pass a function into the p_thread_create function which would be the action that the thread performs. Since the threads that will manage the inputs of the sensors will essentially do the same thing, I decided to use one function and the thread will decide which pointers to update values to, depending on the GPIO pin that it is reading from.

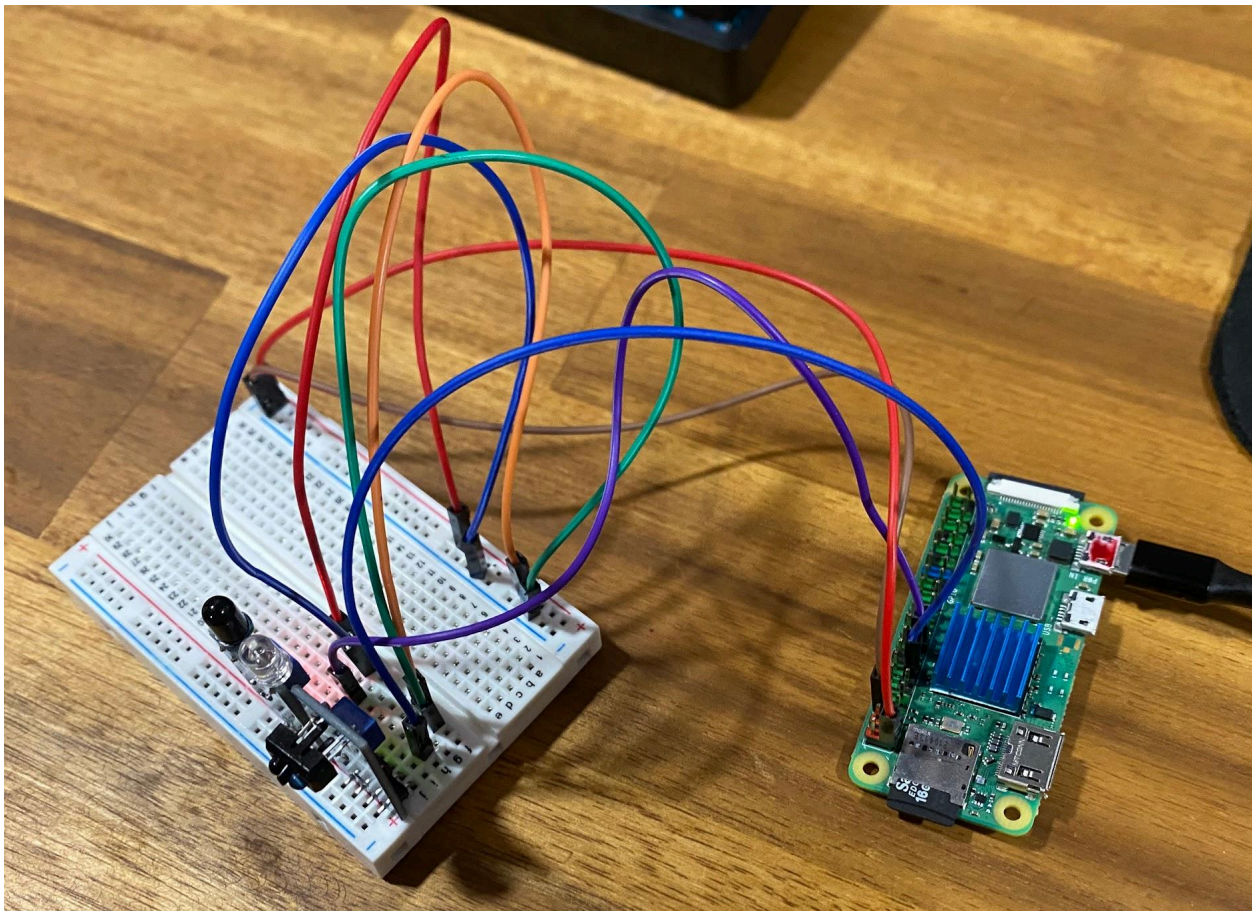
From the past assignment, I was reminded that mutex locks would need to be utilized to prevent race conditions, such as when the main loop checks a value and the respective thread changes the read value. I was going to use one mutex lock that the 2 threads and main loop will use, but I figured that method was pointless since that would be like running the program synchronously. I decided to use two mutex locks, one for each sensor, and each sensor will share their lock with the main loop, so if the main loop was reading the value of one sensor, the other sensor can still update its value since the main loop is not checking it yet.

I decided that for the output of the program I was going to make it so that the output is one continuous screen that updates values instead of a constant stream of print statements to the console. I did this by using ANSI escape codes that allow for more customizable writing to terminal behavior and I took advantage of this by making so that the output of the program stayed static on screen instead of scrolling down.

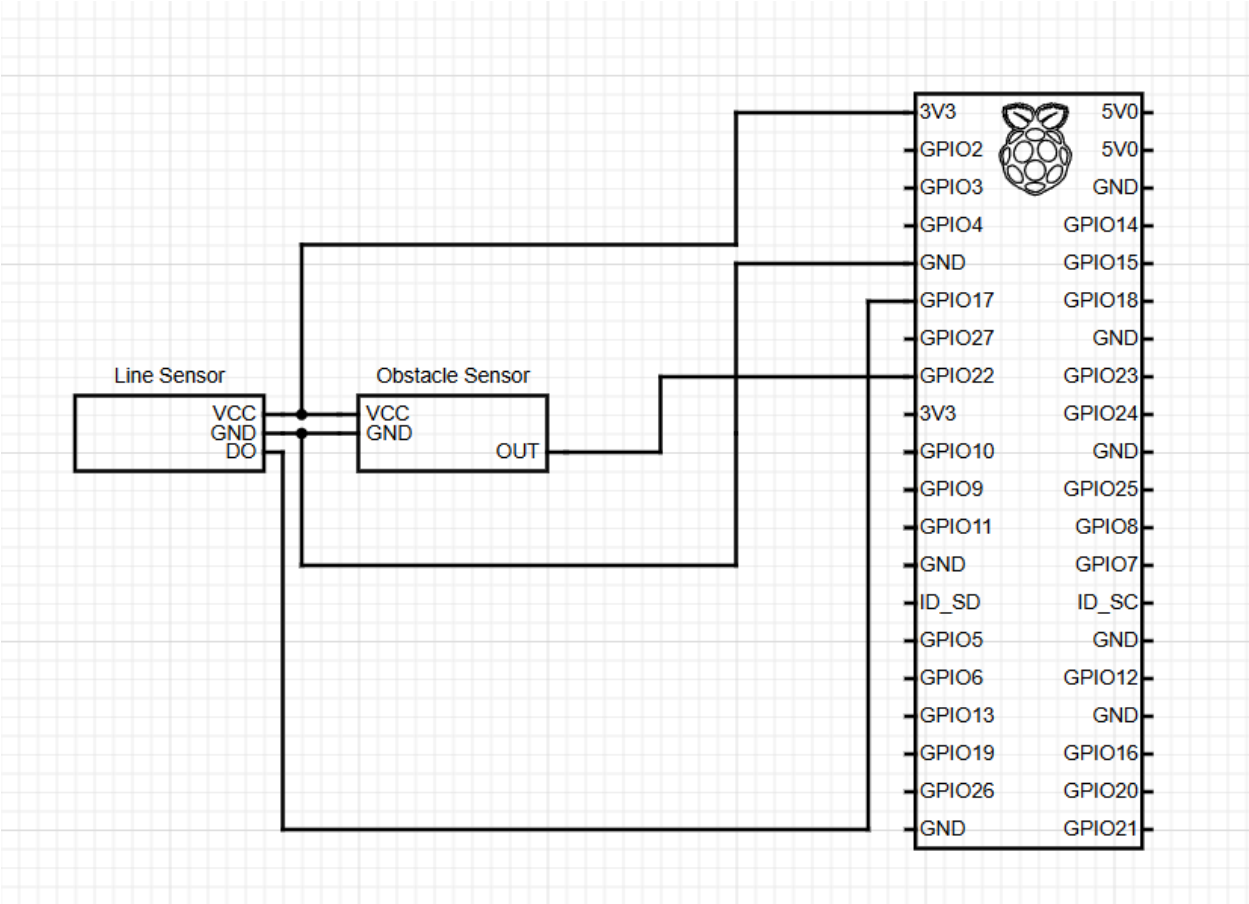
Issues and Resolutions:

I did not have many issues with this assignment since it was more of a refresher on threads. There were some hiccups with threads and making them join back to the main program upon a user interrupt, which caused the program to hang infinitely, but that was resolved with a reboot and the addition of an isProgramRunning flag to make the threads close.

Photo of completed circuit:



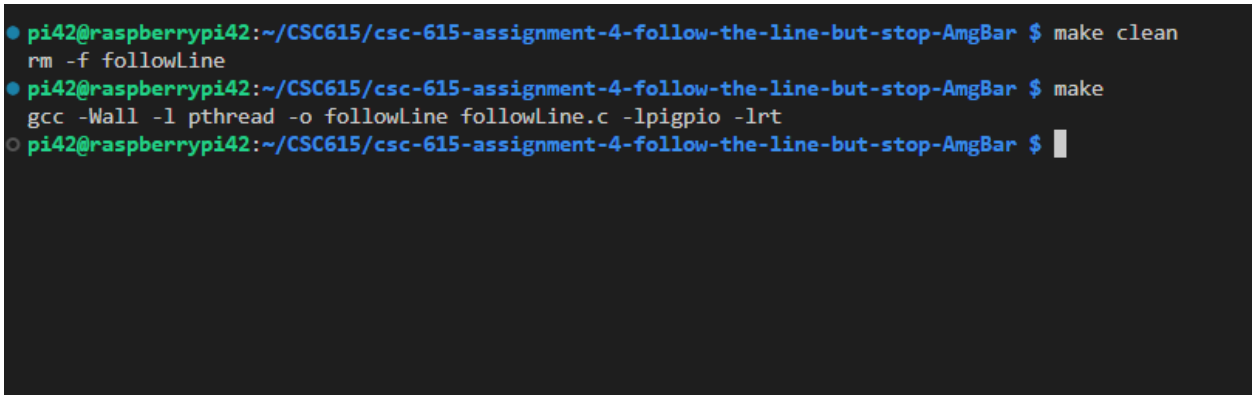
Hardware Diagram:



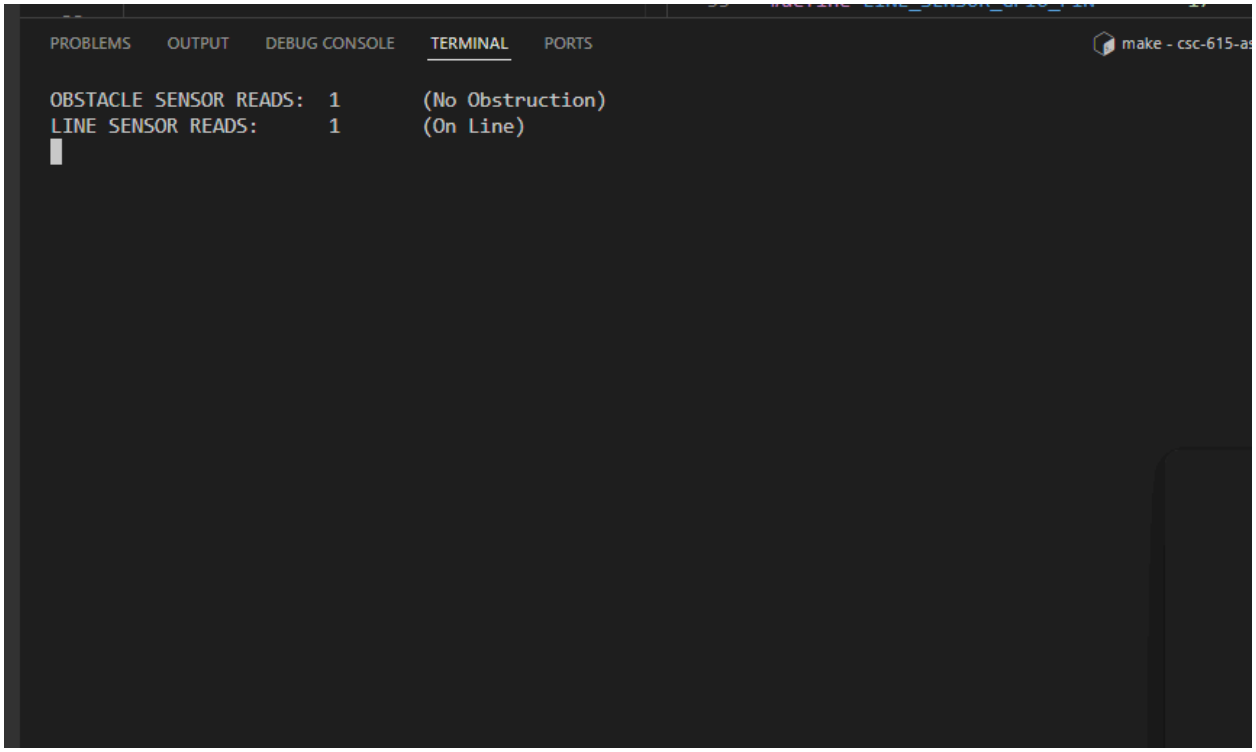
Analysis:

The two sensors are accurate in determining an obstruction and differentiating between white and black. Using my program, quick movements over the sensor are also picked up so I would say that these sensors are accurate for what they were made to do. As for distance, these sensor lack in object detection at a distance, even after tweaking the sensitivity screws, they seem to only work for objects that are about an inch away from the sensors. Even though this is a very small functional distance, I would say that it is enough for something like a self-maneuvering car

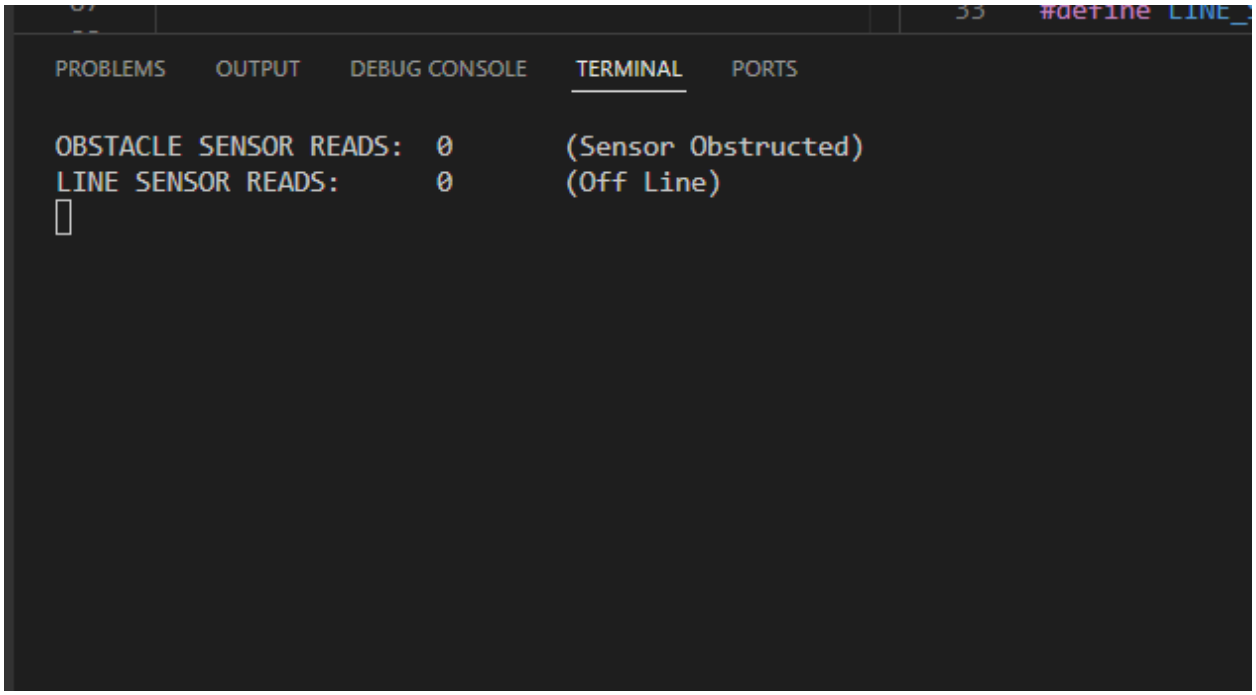
Screen shot of compilation:

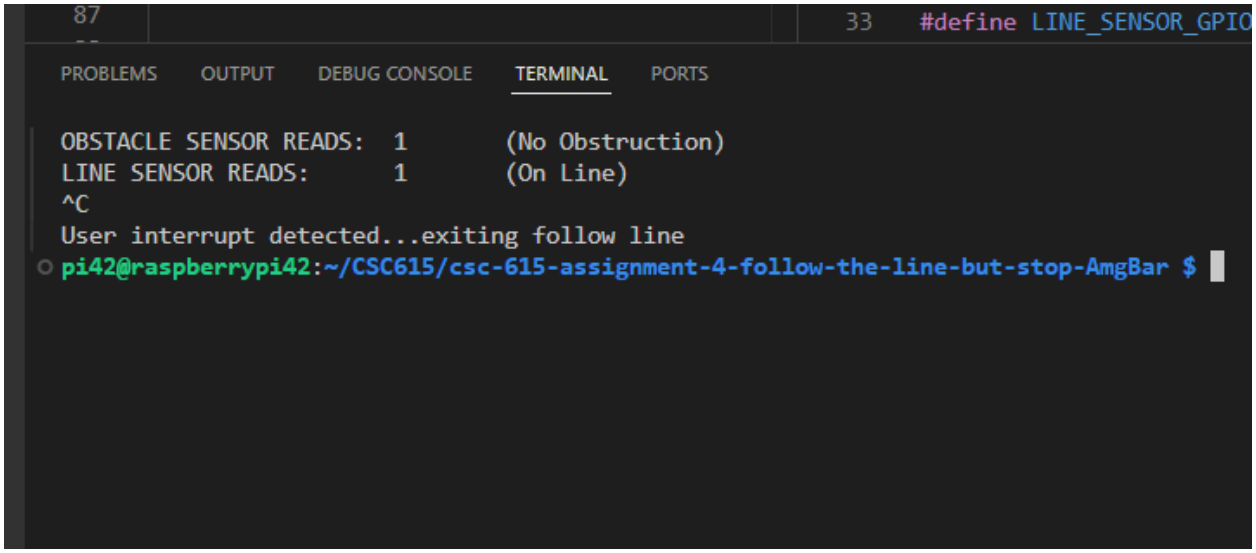


Screen shot(s) of the execution of the program:




PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
make - csc-
OBSTACLE SENSOR READS: 0 (Sensor Obstructed)
LINE SENSOR READS: 1 (On Line)


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
OBSTACLE SENSOR READS: 0 (Sensor Obstructed)
LINE SENSOR READS: 0 (Off Line)


87
33 #define LINE_SENSOR_GPIO
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
OBSTACLE SENSOR READS: 1 (No Obstruction)
LINE SENSOR READS: 1 (On Line)
^C
User interrupt detected...exiting follow line
pi42@raspberrypi42:~/CSC615/csc-615-assignment-4-follow-the-line-but-stop-AmgBar $